

```
In [ ]: NAME:VISHAKHA VIKAS CHAVAN  
        BRANCH:AIDS-A  
        BATCH:B  
        ROLL NO.:EN24107026
```

```
In [ ]: ASSIGNMENT NO : 1  
        Create an image classifier using a traditional machine learning approach Random  
        how minor modifications (like noise) affect its predictions. (dataset: MNIST (ha
```

```
In [1]: import numpy as np  
        import matplotlib.pyplot as plt  
  
        from sklearn.datasets import fetch_openml  
        from sklearn.model_selection import train_test_split  
        from sklearn.ensemble import RandomForestClassifier  
        from sklearn.metrics import accuracy_score, classification_report, confusion_mat
```

```
In [2]: mnist = fetch_openml('mnist_784', version=1, as_frame=False)
```

```
In [3]: X = mnist.data  
        y = mnist.target.astype(int)
```

```
In [4]: # Use a subset for faster execution  
        X_subset = X[:20000]  
        y_subset = y[:20000]
```

```
In [5]: X_train, X_test, y_train, y_test = train_test_split(  
        X_subset,  
        y_subset,  
        test_size=0.2,  
        random_state=42,  
        stratify=y_subset  
        )
```

```
In [6]: # Create Random Forest model  
        model = RandomForestClassifier(  
            n_estimators=100,  
            random_state=42,  
            n_jobs=-1  
        )  
        model.fit(X_train, y_train)
```

```
Out[6]: ▼ RandomForestClassifier ⓘ ?  
        ► Parameters
```

```
In [7]: # Predict original test images  
        y_pred = model.predict(X_test)
```

```
In [8]: # Evaluation  
        print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy: 0.95175

```
In [9]: # Add small random noise
noise_level = 30

noise = np.random.normal(
    loc=0,
    scale=noise_level,
    size=X_test.shape
)

X_test_noisy = X_test + noise

# Keep pixel values within valid range
X_test_noisy = np.clip(X_test_noisy, 0, 255)
```

```
In [10]: # Predict noisy test images
y_pred_noisy = model.predict(X_test_noisy)
```

```
In [11]: # Evaluation
print("noisy_accuracy:", accuracy_score(y_test, y_pred_noisy))

noisy_accuracy: 0.70025
```

```
In [12]: # Compare Accuracies
accuracy_drop = accuracy_score(y_test, y_pred) - accuracy_score(y_test, y_pred_noisy)

print("Clean Image Accuracy :", accuracy_score(y_test, y_pred))
print("Noisy Image Accuracy :", accuracy_score(y_test, y_pred_noisy))
print("Accuracy Drop          :", accuracy_drop)

Clean Image Accuracy : 0.95175
Noisy Image Accuracy : 0.70025
Accuracy Drop          : 0.25149999999999995
```

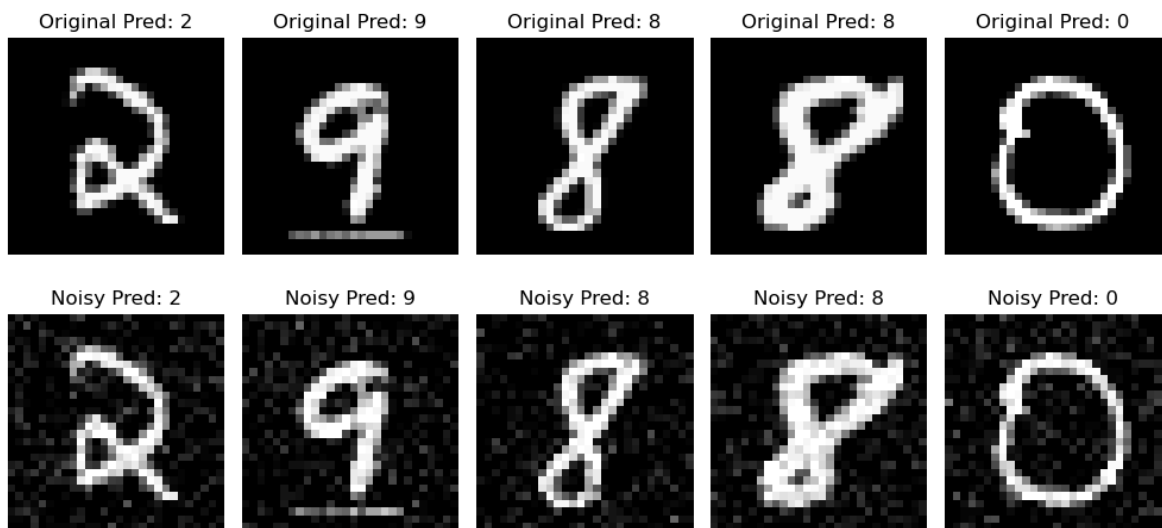
```
In [13]: plt.figure(figsize=(10, 5))

for i in range(5):

    # Original image
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap="gray")
    plt.title("Original Pred: " + str(y_pred[i]))
    plt.axis("off")

    # Noisy image
    plt.subplot(2, 5, i + 6)
    plt.imshow(X_test_noisy[i].reshape(28, 28), cmap="gray")
    plt.title("Noisy Pred: " + str(y_pred_noisy[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
In [14]: print("Classification Report for Original Images:")
print(classification_report(y_test, y_pred))
```

```
Classification Report for Original Images:
              precision    recall  f1-score   support

    0:       0.96         0.99         0.97         399
    1:       0.97         0.98         0.98         456
    2:       0.92         0.94         0.93         386
    3:       0.95         0.92         0.93         415
    4:       0.94         0.95         0.95         389
    5:       0.97         0.96         0.96         355
    6:       0.97         0.97         0.97         394
    7:       0.97         0.95         0.96         419
    8:       0.95         0.91         0.93         384
    9:       0.91         0.95         0.93         403

 accuracy          0.95         0.95         0.95         4000
 macro avg         0.95         0.95         0.95         4000
 weighted avg      0.95         0.95         0.95         4000
```

```
In [15]: print("Classification Report for Noisy Images:")
print(classification_report(y_test, y_pred_noisy))
```

```
Classification Report for Noisy Images:
              precision    recall  f1-score   support

    0:       0.86         0.96         0.91         399
    1:       1.00         0.05         0.10         456
    2:       0.51         0.91         0.66         386
    3:       0.75         0.86         0.80         415
    4:       0.81         0.81         0.81         389
    5:       0.81         0.83         0.82         355
    6:       0.92         0.93         0.92         394
    7:       0.99         0.43         0.60         419
    8:       0.43         0.92         0.58         384
    9:       0.84         0.43         0.57         403

 accuracy          0.70         0.70         0.70         4000
 macro avg         0.79         0.71         0.68         4000
 weighted avg      0.80         0.70         0.67         4000
```